Deployment Manual

Setting up the hiring dashboard from nothing: a spreadsheet, a robot Google account, a mail password, and one deploy. Six phases, about fifty minutes.

No domain required No card required No OAuth consent screen

PHASE 0 What you are building

One Next.js service on Render, talking to three outside things.

SERVICE	ROLE	COST
Google Sheet	The database — candidates, templates, settings, log	Free
Groq	Writes the emails	Free tier
Gmail SMTP	Sends them	Free, ~500/day

There is no separate backend and no other database. The sheet *is* the database — you type into it, the app reads and writes back to it.

PHASE 1

The spreadsheet

10 min

1.1 Create it

New Google Sheet, named anything. Look at the URL and copy the part between `/d/` and `/edit` — that is `SHEET_ID`.

```
https://docs.google.com/spreadsheets/d/1a2B3cD4eF5gH6iJ7kL8mN9oP/edit
                                       └──────────────────────────┘
                                                  SHEET_ID
```

1.2 Service account

A **service account** is a robot Google account. Unlike a normal login it has no consent screen, no browser popup, and no token that expires — it is a key that works until you revoke it. That is why the project uses one.

At console.cloud.google.com (https://console.cloud.google.com):

1. **Create a project** (top-left dropdown → New Project). Name it `hr-automation`.
2. **APIs & Services → Library** → search "Google Sheets API" → **Enable**. Miss this and every call fails with a 403.
3. **IAM & Admin → Service Accounts → Create service account**. Name it `hr-dashboard`. Skip the optional roles and access steps — they do not apply here.
4. Open it → **Keys → Add key → Create new key → JSON**. A `.json` file downloads.

> **That file is a password.** Never commit it, never paste it into a chat or a ticket. It has full write access to every sheet you share with it.

1.3 Share the sheet with the robot

Open the downloaded JSON and find `client_email`:

```
"client_email": "hr-dashboard@hr-automation-412....iam.gserviceaccount.com"
```

Copy that address. In the spreadsheet → **Share** → paste it → **Editor** → Send.

> **This is the step everyone skips.** A service account cannot see a sheet just because you own it — you share it explicitly, exactly as you would with a person. Missing this produces `E-SHEET-PERM`.

PHASE 2

Gmail App Password

5 min

An **App Password** is a 16-character password that logs into Gmail's SMTP server directly, bypassing OAuth entirely.

1. On the sending account, turn on **2-Step Verification** at myaccount.google.com/security (https://myaccount.google.com/security). Without it the App Passwords page does not exist.
2. Go to myaccount.google.com/apppasswords (https://myaccount.google.com/apppasswords). Name it `hr-dashboard` → **Create**.
3. Google shows four groups of four: `abcd efgh ijkl mnop`. **Delete the spaces** → `abcdefghijklmnop`. It is shown once.

> **Personal Gmail only.** Google Workspace accounts cannot use App Passwords — Google forces OAuth 2.0 there. Check the address *before* planning a deploy: if it does not end in `@gmail.com`, either send from a personal Gmail with the company name in `MAIL_FROM`, or point `MAIL_HOST` and `MAIL_PORT` at the real mail server. Nothing in the code is Gmail-specific.

> An App Password grants full access to that mailbox over SMTP and IMAP — send *and* read. Google offers no narrower scope. It is revocable in one click from the same page, it is named so its owner can see what it belongs to, and every send it makes lands in that account's Sent folder. Say this plainly to a client rather than letting them find out later.

2.1 Groq key

console.groq.com/keys (https://console.groq.com/keys) → sign in → **Create API Key**. Free tier, no card.

PHASE 3 Run it locally first

10 min

Do this before deploying. Debugging on your own machine is far faster than debugging through build logs.

```
npm install                 # root: the sheet setup scripts
cd dashboard && npm install # the web app itself
```

Two `package.json` files, two installs — the dashboard deploys from `dashboard/` alone and cannot reach outside that folder, so it carries its own dependencies.

3.1 The env file

```
cp .env.example .env.local   # you are in dashboard/
```

Edit `.env.local`, never `.env.example`. The template is tracked by git; the `.local` file is ignored. Values typed into the template are one `git commit -a` away from being published, and a leaked service-account key must then be revoked and reissued. Nothing reads `.env.example` at runtime, so the symptom is a confusing "SHEET_ID is not set" on a file that visibly has one.

```
SHEET_ID=1a2B3cD4eF5gH6iJ7kL8mN9oP
GOOGLE_SERVICE_ACCOUNT_JSON='{"type":"service_account","project_id":...}'

DASHBOARD_PASSWORD=shared-with-the-hr-team
SESSION_SECRET=<output of: openssl rand -hex 32>

GROQ_API_KEY=gsk_...

MAIL_USER=hiring@example.com
MAIL_PASSWORD=                # ← LEAVE BLANK for now, on purpose
MAIL_FROM=Company Hiring <hiring@example.com>
```

- `GOOGLE_SERVICE_ACCOUNT_JSON` — flatten the downloaded JSON to **one line** and wrap it in **single** quotes, because the JSON itself is full of double quotes.
- `SESSION_SECRET` — signs the login cookie. Any 64 random hex characters; it only has to be unguessable.
- `MAIL_PASSWORD` — **leave blank deliberately.** The app refuses to send without it, which is what you want while still clicking around.

3.2 Build the tabs

```
cd .. && npm run bootstrap:sheets
```

Creates all four tabs with the right headers and seeds the Config defaults. **Idempotent** — safe to re-run any time; it repairs rather than duplicates, and it is the fix for `E-SHEET-SCHEMA`.

```
  • tab "Applicants" created
  • tab "Templates" created
  • tab "EmailLog" created
  • tab "Config" created
  • "Applicants" headers written (15 columns)
  • Config key "dry_run" added (= true)
  ...
✔ Done.
```

Optional, and worth it the first time:

```
npm run seed:demo   # one working template + 3 fake candidates
```

3.3 Start it

```
cd dashboard && npm run dev   # http://localhost:3000
```

Log in with `DASHBOARD_PASSWORD`. You should see the **Inbox** with the seeded candidates.

> With `SHEET_ID` left blank the app runs on a built-in sample dataset instead — fully clickable, nothing saved, nothing sent. Useful for a look around before committing to any Google setup.

PHASE 4

The sheet's shape

You type into five columns. Everything to the right of `stage` is written by the app — leave those cells empty.

APPLICANTS

A	B	C	D	E	F
`applicant_id`	`name`	`email`	`job_role`	`category`	`notes`
APP-1001	Priya Shah	priya@…	Frontend Intern	Intern	met at the PDEU fair, strong React

Then `stage` in column G, and eight app-written columns after it.

- **`applicant_id` must be unique.** Every action resolves a row by it, and a lookup returns the *first* match — so a duplicate id means acting on the second row silently acts on the first. The Inbox flags these with a banner naming the sheet rows, and preflight fails on them. Any format works: `APP-1001`, `1`, `priya-shah`.
- **`notes` is the highest-leverage column.** Free text handed to the model as context. "Met at PDEU fair, strong on React, weak on backend" produces a specific email; an empty cell produces a generic one.

TEMPLATES

Fill `subject` and `html` with merge fields — `{{name}}`, `{{job_role}}`, `{{company_name}}`, `{{hr_signature}}`. Set `is_active` to `TRUE` for a template to be usable, and `is_default` to `TRUE` on exactly one.

Merge fields are **fail-closed**: an unresolved `{{field}}` blocks the send rather than mailing someone the literal text.

CONFIG

Key/value settings, edited on the app's **Settings** page rather than by hand.

KEY	SHIP AS	MEANING
`dry_run`	`true`	Log sends instead of delivering
`toggle_send`	`false`	Master switch for sending
`send_daily_cap`	`400`	Self-imposed, under Gmail's ~500
`company_email`	your address	Becomes the `Reply-To`

PHASE 5

Deploy to Render

10 min

5.1 Push

```
git push origin main
```

The blueprint tracks `main`.

5.2 Blueprint

dashboard.render.com (https://dashboard.render.com) → **New** → **Blueprint** → connect GitHub → pick the repo.

Render reads `render.yaml`, creates one web service, and prompts for each secret. That file already sets the build command, the start command, Node 22, and a health check on `/login` — not `/`, which redirects when signed out and would read as a 307 failure.

5.3 Fill in the secrets

Same values as `.env.local`, with one difference.

VARIABLE	VALUE
`DASHBOARD_PASSWORD`	same
`SESSION_SECRET`	same
`SHEET_ID`	same
`GOOGLE_SERVICE_ACCOUNT_JSON`	**Paste the raw JSON — no quotes, no escaping.** Render's value box is not a shell, so the single quotes needed locally would be read as part of the JSON and break it.
`GROQ_API_KEY`	same
`MAIL_USER`	the sending address
`MAIL_PASSWORD`	**now** paste the 16 characters
`MAIL_FROM`	display name and address

Leave `MAIL_HOST`, `MAIL_PORT` and `GROQ_MODEL` blank — they default to `smtp.gmail.com`, `465`, and `llama-3.1-8b-instant`. First build takes about four minutes.

> The free plan sleeps after 15 minutes idle, so the first request after a quiet spell takes ~30 seconds. Fine for an internal tool.

PHASE 6

Verify, then go live

Do not reorder this. Every step is a chance to catch a problem while it is still harmless.

6.1 Preflight

Open the deployed URL → log in → **Console** → **Run preflight**. Every check must pass: sheet reachable, headers matching the schema, Groq responding, no duplicate ids, no malformed addresses, and the mailbox actually logging in.

That last one opens a real SMTP connection and authenticates — the difference between "a password is set" and "that password works".

If it fails there, it is one of three things, in order of likelihood:

1. The App Password was pasted **with spaces** in it
2. The **normal Google account password** was pasted instead
3. 2-Step Verification is not actually on

6.2 Rehearse with dry run still on

1. Add yourself as a candidate in the Inbox.
2. Draft → Approve → Send.
3. Nothing is delivered; a row appears in the Email Log marked *dry run*.

This proves the whole pipeline — sheet read, template match, merge fields, write-back — with sending physically impossible.

6.3 Go live

1. Settings → **Sending** → on
2. Settings → **dry run** → off
3. **Run preflight again.** The mailbox check is now a hard failure rather than optional.
4. Send one real email **to yourself**.

If it arrives, you are done. Every other send takes the identical code path.

REFERENCE

What protects you once it is live

- **Sending is off by default, twice over** — `dry_run` on *and* `toggle_send` off. Both are deliberate flips.
- **Missing credentials refuse the send** with `E-CONFIG-MISSING` before anything is written. It never logs a send that did not happen.
- **A failed send leaves the row** `APPROVED`, not `SENT`, with the error code in the row. Retry is safe; a candidate cannot be silently lost.
- **Unresolved merge fields block the send** rather than mailing `{{name}}`.
- **Every send lands in the Gmail Sent folder** — a second audit trail, free from using a real mailbox, and the place to check before retrying anything ambiguous.

WHEN SOMETHING BREAKS

CODE	MEANING	FIX
`E-SHEET-PERM`	Robot cannot see the sheet	Share it with `client_email` as Editor
`E-SHEET-SCHEMA`	Columns drifted	`npm run bootstrap:sheets`
`E-CONFIG-MISSING`	Dry run off, no mail credentials	Set `MAIL_PASSWORD` in the environment
`E-MAIL-AUTH`	Login rejected	App Password, no spaces, 2FA on
`E-MAIL-429`	Gmail throttling	~500/day rolling 24h; resumes on its own
`E-MAIL-REJECTED`	Dead address	Fix it in the candidate's Email box
`E-MAIL-NETWORK`	Cannot reach the server	465 needs implicit TLS, 587 needs STARTTLS — a mismatch hangs rather than erroring

BROKEN LOGO IN A RECEIVED EMAIL

Not a fault. Mail clients fetch images through their own proxy servers, so a `localhost` URL is unreachable to them and the `alt` text shows instead. Sends from a local machine always look like this. Deployed, `RENDER_EXTERNAL_URL` resolves it automatically and no configuration is needed. To preview locally, point `COMPANY_LOGO_BASE_URL` at the deployed URL — not the `company_logo_url` setting, which lives in the shared sheet and would follow you into production.

ROTATING A SECRET

Same for all of them: revoke at the source, generate a new one, update the environment variable, redeploy. App Password → delete at myaccount.google.com/apppasswords (https://myaccount.google.com/apppasswords). Service account key → delete under **Keys**. Never edit a secret by committing it.

APPENDIX

Deploying for someone else

Only two credentials genuinely require the client's account. The rest you own and reuse across every deployment.

CREDENTIAL	THEIR ACCOUNT?	WHO
`MAIL_PASSWORD`	Yes — unavoidable	them
`SHEET_ID` + sheet shared	Yes — one click	them
`GOOGLE_SERVICE_ACCOUNT_JSON`	No	you
`GROQ_API_KEY`	No	you
`DASHBOARD_PASSWORD`	No	you
`SESSION_SECRET`	No	you

The service account is the non-obvious one: it does not have to live in their Google account. It is a robot address that needs Editor access to their sheet, so you create it once under your own project and they only ever share a spreadsheet with it. That removes an entire Cloud Console walkthrough from the session, leaving about eight minutes of their time.

Have them create the spreadsheet **in their own Drive**. If it sits in yours, their hiring history leaves with you.

> **Never let an App Password travel through chat.** Have the deployment environment open and type it straight into the variable while you are with them. If it must be transported, use a password manager's one-time link — never messaging or plain email. And strip the spaces while they can still re-read it: it is shown once, and a wrong paste means booking them again.

COLLECTION SHEET

CLIENT / DATE

CLIENT _____

DATE _____

FROM THEM

MAIL_USER _____

MAIL_PASSWORD (16 ch, no spaces) _____

SHEET_ID _____

☐ shared with robot as EDITOR

MAIL_FROM (display name) _____

company_email (Reply-To) _____

ALREADY YOURS

☐ GOOGLE_SERVICE_ACCOUNT_JSON

☐ GROQ_API_KEY

☐ DASHBOARD_PASSWORD

☐ SESSION_SECRET

CHECK THE PASSWORD BEFORE THEY WALK AWAY

Ten seconds, sends nothing, and tests the one credential you cannot regenerate without them.

```
cd dashboard
MAIL_USER=their@gmail.com MAIL_PASSWORD=abcdefghijklmnop \
node -e "require('nodemailer').createTransport({host:'smtp.gmail.com',
port:465,secure:true,auth:{user:process.env.MAIL_USER,
pass:process.env.MAIL_PASSWORD}}).verify().then(()=>console.log('works'))
.catch(e=>console.log('FAILED',e.message))"
```

A `535-5.7.8` means ask them to generate another one now.
